UNIVERSITY
GWALIOR • MP • INDIA
CELEBRATING DREAM

ITM UNIVERSITY,GWALIOR.

Practical file (Hands on lab)

Machine Learning (CSE0314)

SUBMITTED TO :
Mr. Subham Dhakary
Asst. Professor,
Dept of CSA, SOCIS .

SUBMITTED BY:
Mohammad Jubaid
BETN1AI25080
AIML Section E

S.NO	TITLE	PG NO	Signature
1.	Load the Adult Income dataset using Pandas and display the first 10 records, last 10 records, dataset information, descriptive statistics, and identify missing values.	1-2	
2.	Perform data preprocessing by handling missing values, encoding categorical features, and applying feature scaling using StandardScaler or MinMaxScaler.	2-4	
3.	Split the dataset into Training (70%), Validation (15%), and Testing (15%) datasets and display the shape of each dataset.	4-5	
4.	Apply Feature Selection using Correlation Matrix and SelectKBest. Compare the selected features and identify the most important attributes.	5-6	
5.	Compare the dataset before and after preprocessing using suitable visualizations and summarize how preprocessing improved the dataset quality.	7-9	

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
url="https://raw.githubusercontent.com/mohammadjubaid76-collab/adult_income_ml/main/adult.csv"
df=pd.read_csv(url, na_values="?")
df
```

	age	workclass	fnlwgt	education	education.num	marital.status	occupation	relationship	race	sex	capital.gai
0	90	NaN	77053	HS-grad	9	Widowed	NaN	Not-in-family	White	Female	
1	82	Private	132870	HS-grad	9	Widowed	Exec-managerial	Not-in-family	White	Female	
2	66	NaN	186061	Some-college	10	Widowed	NaN	Unmarried	Black	Female	
3	54	Private	140359	7th-8th	4	Divorced	Machine-op-inspct	Unmarried	White	Female	
4	41	Private	264663	Some-college	10	Separated	Prof-specialty	Own-child	White	Female	
...	...	...	...	...	...	...	...	...	...	...	
32556	22	Private	310152	Some-college	10	Never-married	Protective-serv	Not-in-family	White	Male	
32557	27	Private	257302	Assoc-acdm	12	Married-civ-spouse	Tech-support	Wife	White	Female	
32558	40	Private	154374	HS-grad	9	Married-civ-spouse	Machine-op-inspct	Husband	White	Male	
32559	58	Private	151910	HS-grad	9	Widowed	Adm-clerical	Unmarried	White	Female	
32560	22	Private	201490	HS-grad	9	Never-married	Adm-clerical	Own-child	White	Male	

32561 rows × 15 columns

```
print("First 10 records:",df.head(10))
print("Last 10 records:",df.tail(10))
print("Dataset information:",df.info())
print("Descriptive statistics:",df.describe(include="all"))
```

```
First 10 records:    age    workclass  fnlwgt     education  education.num marital.status  \
0   90          NaN   77053       HS-grad             9        Widowed
1   82      Private  132870       HS-grad             9        Widowed
2   66          NaN  186061  Some-college            10        Widowed
3   54      Private  140359       7th-8th             4       Divorced
4   41      Private  264663  Some-college            10      Separated
5   34      Private  216864       HS-grad             9       Divorced
6   38      Private  150601          10th             6      Separated
7   74    State-gov   88638     Doctorate            16  Never-married
8   68  Federal-gov  422013       HS-grad             9       Divorced
9   41      Private   70037  Some-college            10  Never-married

          occupation    relationship   race     sex  capital.gain  \
0                NaN   Not-in-family  White  Female             0
1    Exec-managerial   Not-in-family  White  Female             0
2                NaN       Unmarried  Black  Female             0
3  Machine-op-inspct       Unmarried  White  Female             0
4     Prof-specialty       Own-child  White  Female             0
5      Other-service       Unmarried  White  Female             0
6       Adm-clerical       Unmarried  White    Male             0
7     Prof-specialty  Other-relative  White  Female             0
8     Prof-specialty   Not-in-family  White  Female             0
9       Craft-repair       Unmarried  White    Male             0

   capital.loss  hours.per.week native.country income
0          4356              40  United-States  <=50K
1          4356              18  United-States  <=50K
2          4356              40  United-States  <=50K
3          3900              40  United-States  <=50K
4          3900              40  United-States  <=50K
5          3770              45  United-States  <=50K
6          3770              40  United-States  <=50K
7          3683              20  United-States   >50K
8          3683              40  United-States  <=50K
9          3004              60            NaN   >50K
Last 10 records:        age         workclass  fnlwgt     education  education.num  \
```

```
32551   43  Self-emp-not-inc   27242  Some-college            10
32552   32           Private   34066          10th             6
32553   43           Private   84661     Assoc-voc            11
32554   32           Private  116138       Masters            14
32555   53           Private  321865       Masters            14
32556   22           Private  310152  Some-college            10
32557   27           Private  257302    Assoc-acdm            12
32558   40           Private  154374       HS-grad             9
32559   58           Private  151910       HS-grad             9
32560   22           Private  201490       HS-grad             9

           marital.status         occupation   relationship  \
32551  Married-civ-spouse       Craft-repair        Husband
32552  Married-civ-spouse  Handlers-cleaners        Husband
32553  Married-civ-spouse              Sales        Husband
32554       Never-married       Tech-support  Not-in-family
32555  Married-civ-spouse    Exec-managerial        Husband
32556       Never-married    Protective-serv  Not-in-family
32557  Married-civ-spouse       Tech-support           Wife
32558  Married-civ-spouse  Machine-op-inspct        Husband
32559             Widowed       Adm-clerical      Unmarried
```

```
df.isnull().sum()
```

```
age                0
workclass       1836
fnlwgt             0
education          0
education.num      0
marital.status     0
occupation      1843
relationship       0
race               0
sex                0
capital.gain       0
capital.loss       0
hours.per.week     0
native.country   583
income             0
dtype: int64
```

2. Data Preprocessing

```
categorical_missing = ['workclass', 'occupation', 'native.country']

for col in categorical_missing:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```
df.isnull().sum()
```

```
age              0
workclass        0
fnlwgt           0
education        0
education.num    0
marital.status   0
occupation       0
relationship     0
race             0
sex              0
capital.gain     0
capital.loss     0
hours.per.week   0
native.country   0
income           0
dtype: int64
```

```
categorical_cols = df.select_dtypes(include='object').columns

print(categorical_cols)
```

```
Index(['workclass', 'education', 'marital.status', 'occupation',
       'relationship', 'race', 'sex', 'native.country', 'income'],
      dtype='str')
C:\Users\DELL\AppData\Local\Temp\ipykernel_8232\1616883611.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes ar
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to
  categorical_cols = df.select_dtypes(include='object').columns
```

```
X = df.drop('income', axis=1)
y = df['income']

categorical_cols = X.select_dtypes(include='object').columns
```

```
X = pd.get_dummies(
    X,
    columns=categorical_cols,
    drop_first=True,
    dtype=int
)

print("Shape after encoding:", X.shape)
X.head()
```

```
Shape after encoding: (32561, 97)
C:\Users\DELL\AppData\Local\Temp\ipykernel_8232\710248711.py:4: Pandas4Warning: For backward compatibility, 'str' dtypes are
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to
  categorical_cols = X.select_dtypes(include='object').columns
```

	age	fnlwgt	education.num	capital.gain	capital.loss	hours.per.week	workclass_Local-gov	workclass_Never-worked	workclass_Pri
0	90	77053	9	0	4356	40	0	0	
1	82	132870	9	0	4356	18	0	0	
2	66	186061	10	0	4356	40	0	0	
3	54	140359	4	0	3900	40	0	0	
4	41	264663	10	0	3900	40	0	0	

5 rows × 97 columns

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

numeric_cols = X.select_dtypes(include=['int64', 'float64']).columns
X[numeric_cols] = scaler.fit_transform(X[numeric_cols])

print("Scaled numerical features:")
print(X[numeric_cols].head())
```

```
Scaled numerical features:
        age    fnlwgt  education.num  capital.gain  capital.loss  \
0  3.769612 -1.067997      -0.420060      -0.14592     10.593507
1  3.183112 -0.539169      -0.420060      -0.14592     10.593507
2  2.010110 -0.035220      -0.031360      -0.14592     10.593507
3  1.130359 -0.468215      -2.363558      -0.14592      9.461864
4  0.177296  0.709482      -0.031360      -0.14592      9.461864

   hours.per.week  workclass_Local-gov  workclass_Never-worked  \
0       -0.035429            -0.262097               -0.014664
1       -1.817204            -0.262097               -0.014664
2       -0.035429            -0.262097               -0.014664
3       -0.035429            -0.262097               -0.014664
4       -0.035429            -0.262097               -0.014664

   workclass_Private  workclass_Self-emp-inc  ...  native.country_Portugal  \
0            0.57209               -0.188389  ...                -0.033729
1            0.57209               -0.188389  ...                -0.033729
2            0.57209               -0.188389  ...                -0.033729
3            0.57209               -0.188389  ...                -0.033729
4            0.57209               -0.188389  ...                -0.033729

   native.country_Puerto-Rico  native.country_Scotland  native.country_South  \
0                   -0.059274                -0.019201             -0.049628
1                   -0.059274                -0.019201             -0.049628
2                   -0.059274                -0.019201             -0.049628
3                   -0.059274                -0.019201             -0.049628
4                   -0.059274                -0.019201             -0.049628

   native.country_Taiwan  native.country_Thailand  \
0              -0.039607                -0.023518
1              -0.039607                -0.023518
2              -0.039607                -0.023518
3              -0.039607                -0.023518
4              -0.039607                -0.023518

   native.country_Trinadad&Tobago  native.country_United-States  \
0                        -0.024163                      0.307208
1                        -0.024163                      0.307208
2                        -0.024163                      0.307208
3                        -0.024163                      0.307208
4                        -0.024163                      0.307208

   native.country_Vietnam  native.country_Yugoslavia
0               -0.045408                  -0.022173
1               -0.045408                  -0.022173
```

```
2               -0.045408                -0.022173
3               -0.045408                -0.022173
4               -0.045408                -0.022173

[5 rows x 97 columns]
```

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
    random_state=42
)

print("Training set:", X_train.shape)
print("Validation set:", X_val.shape)
print("Testing set:", X_test.shape)
```

```
Training set: (22792, 97)
Validation set: (4884, 97)
Testing set: (4885, 97)
```

```
preprocessed_df = X.copy()
preprocessed_df['income'] = y.values

preprocessed_df.to_csv("adult_income_preprocessed.csv", index=False)

print("CSV file created successfully!")
```

```
CSV file created successfully!
```

4. Feature Selection using Correlation Matrix

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train)

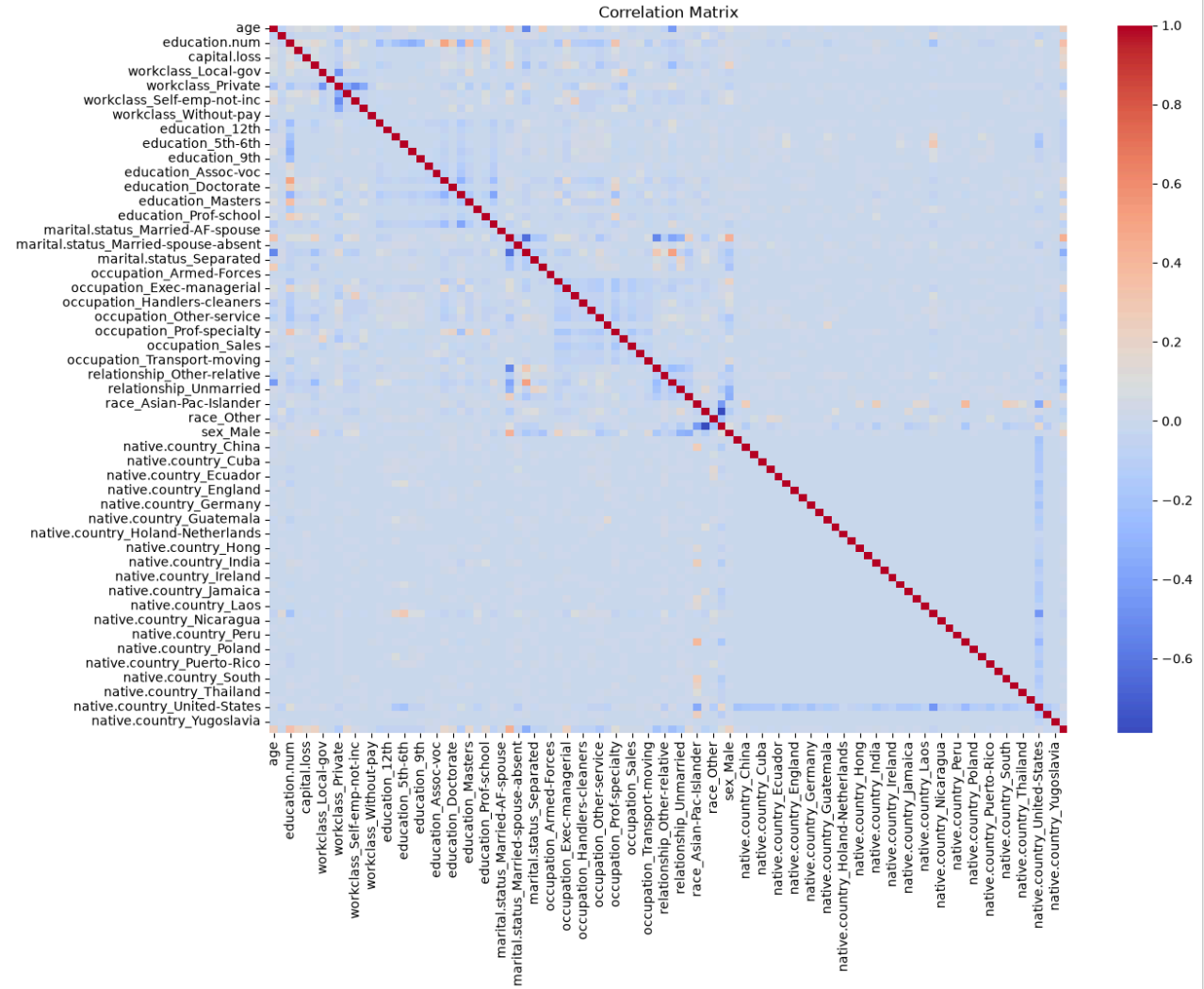

corr_df = X_train.copy()
corr_df['income'] = y_train_encoded


corr_matrix = corr_df.corr()


plt.figure(figsize=(14, 10))
sns.heatmap(corr_matrix, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()


target_corr = (
    corr_matrix['income']
    .drop('income')
    .abs()
    .sort_values(ascending=False)
)

print("Top 5 features by absolute correlation with income:")
print(target_corr.head(5))
```




```
Top 5 features by absolute correlation with income:
marital.status_Married-civ-spouse    0.448304
education.num                        0.337458
marital.status_Never-married         0.320187
age                                  0.234856
relationship_Own-child               0.228089
Name: income, dtype: float64
```

4.1 Feature Selection using SelectKBest

```
from sklearn.feature_selection import SelectKBest, f_classif

# Select the top 5 features
selector = SelectKBest(score_func=f_classif, k=5)

X_train_selected = selector.fit_transform(
    X_train,
    y_train_encoded
)

selected_features = X_train.columns[selector.get_support()]

print("Top 5 features using SelectKBest:")
```

```
print(list(selected_features))

# Display feature scores
feature_scores = pd.DataFrame({
    'Feature': X_train.columns,
    'Score': selector.scores_
}).sort_values(by='Score', ascending=False)

print("\nFeature scores:")
print(feature_scores.head(10))
```

```
Top 5 features using SelectKBest:
['age', 'education.num', 'marital.status_Married-civ-spouse', 'marital.status_Never-married', 'relationship_Own-child']

Feature scores:
                             Feature        Score
29  marital.status_Married-civ-spouse  5732.305619
2                     education.num  2928.798065
31       marital.status_Never-married  2603.313814
0                              age  1330.424093
49             relationship_Own-child  1250.702331
3                      capital.gain  1204.904916
5                     hours.per.week  1202.430581
36         occupation_Exec-managerial  1136.018459
56                         sex_Male  1071.730002
47         relationship_Not-in-family   890.344090
```

```
top_corr_features = list(target_corr.head(5).index)
selectkbest_features = list(selected_features)

common_features = set(top_corr_features).intersection(selectkbest_features)

print("Top 5 Correlation features:")
print(top_corr_features)

print("\nTop 5 SelectKBest features:")
print(selectkbest_features)

print("\nCommon important features:")
print(list(common_features))
```

```
Top 5 Correlation features:
['marital.status_Married-civ-spouse', 'education.num', 'marital.status_Never-married', 'age', 'relationship_Own-child']

Top 5 SelectKBest features:
['age', 'education.num', 'marital.status_Married-civ-spouse', 'marital.status_Never-married', 'relationship_Own-child']

Common important features:
['marital.status_Never-married', 'age', 'marital.status_Married-civ-spouse', 'education.num', 'relationship_Own-child']
```

5. Compare Dataset Before and After Preprocessing

```
df_original = pd.read_csv("adult.csv")
df_original = df_original.replace('?', pd.NA)

print("Original dataset shape:", df_original.shape)
print("Processed dataset shape:", df.shape)
```

```
Original dataset shape: (32561, 15)
Processed dataset shape: (32561, 15)
```

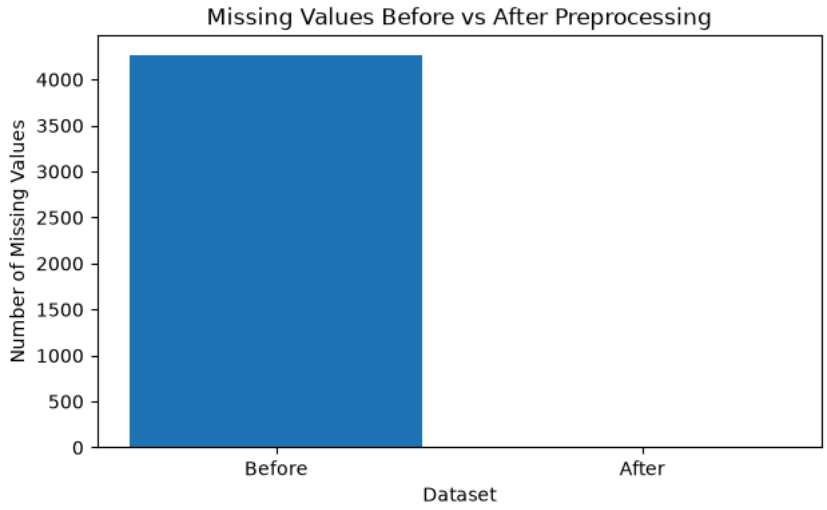
```
before_missing = df_original.isna().sum().sum()
after_missing = df.isna().sum().sum()

print("Missing values before preprocessing:", before_missing)
print("Missing values after preprocessing:", after_missing)

plt.figure(figsize=(7, 4))
plt.bar(
    ["Before", "After"],
    [before_missing, after_missing]
)
plt.title("Missing Values Before vs After Preprocessing")
plt.xlabel("Dataset")
plt.ylabel("Number of Missing Values")
```

```
plt.show()
```

```
Missing values before preprocessing: 4262
Missing values after preprocessing: 0
```

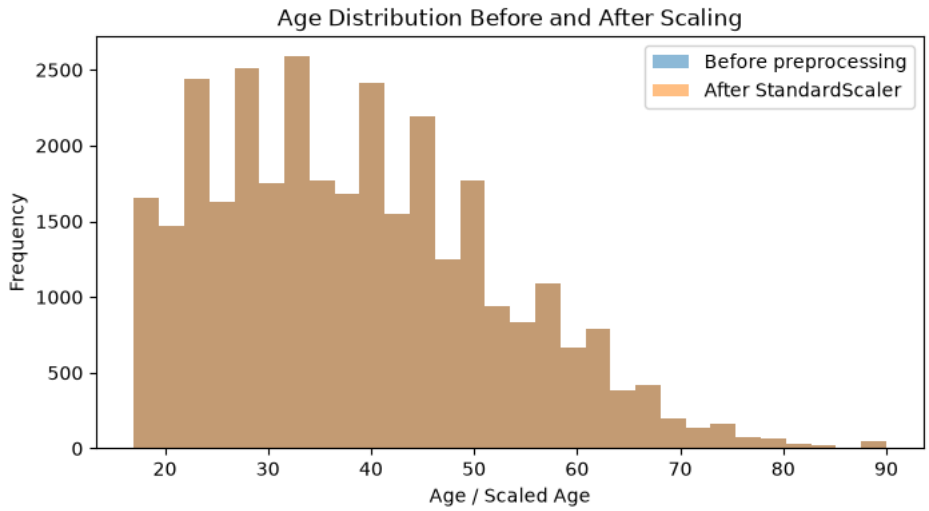



```
plt.figure(figsize=(8,4))

plt.hist(
    df_original['age'].dropna(),
    bins=30,
    alpha=0.5,
    label='Before preprocessing'
)

plt.hist(
    df['age'],
    bins=30,
    alpha=0.5,
    label='After StandardScaler'
)

plt.title('Age Distribution Before and After Scaling')
plt.xlabel('Age / Scaled Age')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```

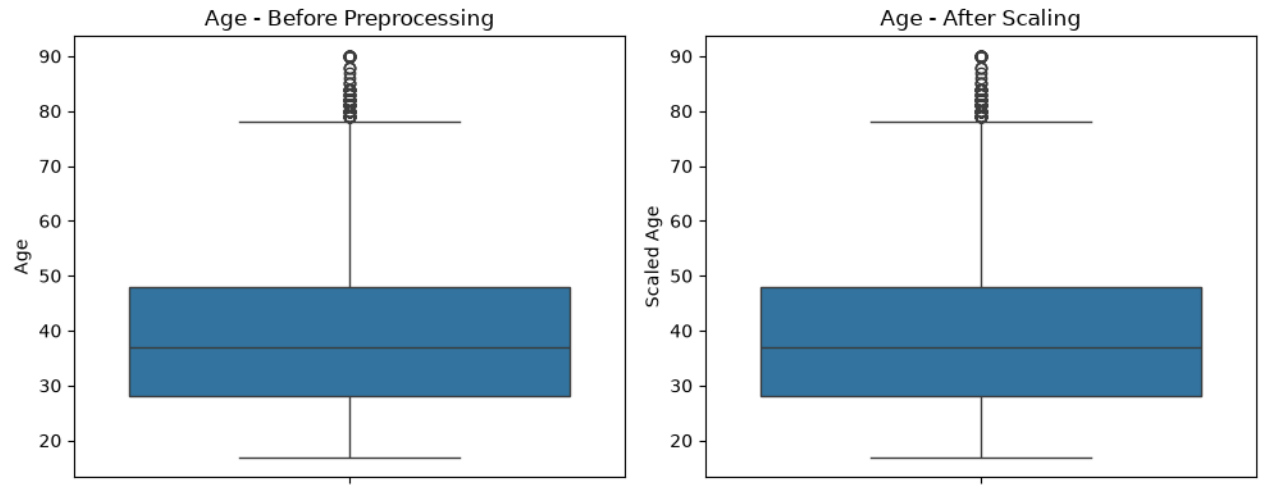



```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

sns.boxplot(y=df_original['age'].dropna(), ax=axes[0])
axes[0].set_title('Age - Before Preprocessing')
axes[0].set_ylabel('Age')

sns.boxplot(y=df['age'], ax=axes[1])
axes[1].set_title('Age - After Scaling')
axes[1].set_ylabel('Scaled Age')
```

```
plt.tight_layout()
plt.show()
```




```
categorical_cols_before = df_original.select_dtypes(include='object').columns
before_count = len(categorical_cols_before)


encoded_cols = [
    col for col in X.columns
    if any(col.startswith(cat + "_") for cat in categorical_cols_before)
]

after_count = len(encoded_cols)

print("Categorical columns before encoding:")
print(list(categorical_cols_before))

print("\nEncoded columns after encoding:")
print(encoded_cols)

print("\nBefore encoding:", before_count)
print("After encoding:", after_count)


plt.figure(figsize=(8, 5))

plt.bar(
    ["Before Encoding", "After Encoding"],
    [before_count, after_count]
)

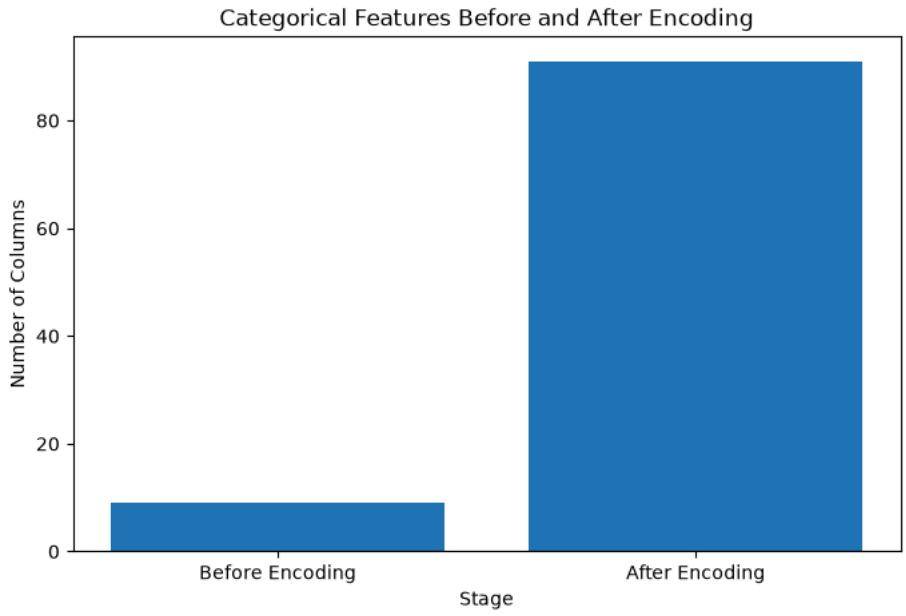
plt.title("Categorical Features Before and After Encoding")
plt.xlabel("Stage")
plt.ylabel("Number of Columns")

plt.show()
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_296\4199439995.py:4: Pandas4Warning: For backward compatibility, 'str' dtypes are
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to
  categorical_cols_before = df_original.select_dtypes(include='object').columns
Categorical columns before encoding:
['workclass', 'education', 'marital.status', 'occupation', 'relationship', 'race', 'sex', 'native.country', 'income']

Encoded columns after encoding:
['workclass_Local-gov', 'workclass_Never-worked', 'workclass_Private', 'workclass_Self-emp-inc', 'workclass_Self-emp-not-inc

Before encoding: 9
After encoding: 91
```



Conclusion

The Adult Income Prediction project was successfully completed using a basic machine learning workflow. The dataset was first loaded and explored to understand its structure and important features. Data cleaning was performed by handling duplicate records and missing values represented by ?. Categorical variables were converted into numerical values using One-Hot Encoding, while numerical features were standardized using StandardScaler. The dataset was then divided into training, validation, and testing sets. Select Best was used for feature selection to identify the most useful features for income prediction. Finally, Logistic Regression was implemented to classify individuals into different income categories. The model was evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix on the validation and test datasets. Overall, the project provided a clear understanding of the complete classification process, from data preprocessing to model evaluation. It also helped in understanding how feature selection and scaling can improve the preparation of data for machine learning.